

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: АЛГОРИТМ АХО-КОРАСИК

Студент гр. 4345

Пикалев Н.Г.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Ахо-Корасик.

Задание.

1. Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая – число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел – i p , где i – позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Входные данные:

NTAG

3

TAGT

TAG

T

Выходные данные:

2 2

2 3

2. Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному

содержащему шаблону образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте *xabvссbababсах*.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Входные данные:

ACTANCA

A\$\$\$A\$

\$

Выходные данные:

1

Вариант 1 – На месте джокера может быть любой символ за исключением заданного.

Основные теоретические положения.

Алгоритм Ахо-Корасик – это высокоэффективный алгоритм поиска вхождений всех строк-образцов в заданную строку. Суть алгоритма заключается в использовании бора и построения по нему конечного детерминированного автомата.

Бор – структура данных, предназначенная для хранения строк. Представляет собой дерево, на рёбрах которого присутствуют буквенные символы. Детерминированный конечный автомат – математическая модель абстрактного вычислительного устройства, которое находится в один момент времени строго в одном состоянии из конечного набора и переходит в другое состояние по точно заданному правилу при получении входного символа.

Выполнение работы.

Наивный метод поиска вхождений строк-образцов в заданную строку занимает время $O(N * M)$, где N и M – длины текста и шаблона. Для больших текстов данный алгоритм будет выполняться слишком долго. Эффективным методом решения такой задачи является алгоритм Ахо-Корасик, позволяющий находить все вхождения строк-образцов за линейное время путём построения конечного автомата.

Для выполнения первой части работы был создан файл lb5.1.py. Он представляет собой программу поиска набора заданных образцов в тексте. В основе программы лежит построение бора из всех шаблонов. Бор хранится в виде массива trie, где каждый узел содержит массив переходов по символам алфавита. Алфавит формируется динамически на основе символов, встречающихся в тексте и шаблонах. Каждый шаблон вставляется в бор посимвольно: если переход по текущему символу отсутствует, создаётся новый узел; если существует – осуществляется переход по нему. В терминальном узле каждого шаблона сохраняется его идентификатор в массиве output.

После построения бора суффиксные и конечные ссылки вычисляются итеративно с помощью обхода в ширину (BFS) с использованием очереди. В процессе обхода бор дополняется до полноценного детерминированного конечного автомата (ДКА): отсутствующие переходы заполняются переходами через суффиксные ссылки, что позволяет выполнять переход по любому символу за $O(1)$ без дополнительных откатов.

Процесс поиска опирается на два типа связей:

1. Суффиксные ссылки – указывают на узел, являющийся наибольшим собственным суффиксом текущей строки. Благодаря им автомат может не возвращаться назад по тексту при несоответствии символа и перейти к проверке других шаблонов.

2. Конечные ссылки – указывают на ближайший по цепочке суффиксных ссылок узел, который является концом какого-либо шаблона.

Благодаря этому можно находить «вложенные» слова, пропуская нетерминальные узлы и избегая лишних подъёмов по автомату.

Переходы по автомату осуществляются через массив `trie`. Поскольку автомат достроен до ДКА, для каждого символа текста переход выполняется за одно обращение к массиву. Если в текущем узле обнаруживается конец шаблона, либо по цепочке конечных ссылок достижимы терминальные узлы, программа вычисляет стартовую позицию вхождения, сохраняет её и в конце выводит отсортированный по позициям и номеру список всех найденных совпадений.

Для выполнения второй части работы был создан файл `lb5.2.py`. Он представляет собой алгоритм поиска одного образца с джокером, который может заменять любой одиночный символ алфавита, за исключением заданного запретного символа.

В начале работы программы шаблон разбивается по символу джокера на набор безмасочных фрагментов. Для каждого фрагмента запоминается его смещение относительно начала исходного шаблона. Бор и автомат строятся из этих фрагментов аналогично первой программе: итеративное построение бора, вычисление суффиксных и конечных ссылок через BFS, достройка до ДКА.

Процесс обхода текста аналогичен первой программе: переход между состояниями автомата выполняется за $O(1)$ через массив `trie`, а для выявления «вложенных» совпадений используются конечные ссылки.

Когда автомат обнаруживает конец какого-либо фрагмента, программа вычисляет предполагаемую позицию начала всего шаблона в тексте: от текущего индекса в тексте отнимается длина фрагмента и вычитается его начальное смещение в шаблоне. Если полученная позиция корректна (не выходит за границы текста), найденный фрагмент сохраняется в массив `found_parts`, где каждый элемент представляет собой список пар (индекс фрагмента, позиция в тексте) для соответствующей предполагаемой позиции начала шаблона.

После завершения обхода текста программа проверяет кандидатов. Для каждой предполагаемой позиции начала шаблона проверяются два условия. Во-первых, все ли безмасочные фрагменты найдены и расположены ли они на правильных позициях относительно начала шаблона. Во-вторых, не попадает ли запретный символ в те позиции текста, которые соответствуют джокерам в шаблоне. Позиции, удовлетворяющие обоим условиям, выводятся как результат (в 1-базовой нумерации).

Затраты алгоритмов по времени и памяти:

Поиск набора образцов: $O(N + M)$ по времени, $O(M)$ по памяти. N – длина текста, M – сумма длин всех шаблонов.

Точный поиск для одного образца с джокером: $O(N + M + K)$ по времени, $O(N + M)$ по памяти. N – длина текста, M – сумма длин всех безмасочных фрагментов, K – длина шаблона.

Исходный код программ см. в приложении А.

Результаты тестирования программ см. в таблице 1 и 2.

Таблица 1 – Тестирование программы lb5.1.py

Ввод	Результат	Комментарий
NTAG 3 TAGT TAG T	2 2 2 3	Пример из условия задачи
ACDTNN 1 DC	(Вхождения отсутствуют)	
CGGACTA 5 GAC GACTA GGG	2 5 3 1 3 2	5 шаблонов

ACTAC GG		
NNNNN	2 1	Одинаковые буквы в тексте и шаблонах
3	2 2	
NNN	2 3	
NN	3 1	
N	3 2	
	3 3	
	4 2	
	4 3	
	5 3	

Таблица 2 – Тестирование программы lb5.2.py

Ввод	Результат	Комментарий
ACTANCA A\$\$\$ \$H	1	Пример из условия задачи
ACTANCA A\$\$\$ \$N	(Совпадений не найдено)	Пример с запрещённым символом
CGNNGN \$N \$C	2 3 5	Нахождение совпадений с запрещённым символом
CCCCC \$C\$ \$D	1 2 3	Одинаковые буквы в тексте

Выводы.

В ходе выполнения данной лабораторной работы был изучен алгоритм Ахо-Корасик, реализованы программы поиска набора образцов и поиска одного образца с джокером. Обе программы выполняются корректно.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb5.1.py

```
from collections import deque

def solve():
    T = input("Введите текст: ").strip().upper()
    n = int(input("Введите число шаблонов: ").strip())
    patterns = [input().strip().upper() for _ in range(n)]

    alphabet = sorted(list(set(T + ''.join(patterns))))
    char_to_idx = {ch: i for i, ch in enumerate(alphabet)}
    sigma = len(alphabet)

    trie = [[-1] * sigma]
    fail = [0]
    output = [[]]
    term = [0]
    node_to_str = [""]

    print("ПОСТРОЕНИЕ БОРА.")
    for idx, pat in enumerate(patterns):
        print(f"\nДобавляем шаблон '{pat}' (ID: {idx + 1}):")
        node = 0
        for ch in pat:
            c_idx = char_to_idx[ch]
            curr_path = node_to_str[node]
            if trie[node][c_idx] == -1:
                trie[node][c_idx] = len(trie)
                trie.append([-1] * sigma)
                fail.append(0)
                output.append([])
                term.append(0)
                node_to_str.append(curr_path + ch)
                print(f"        Создана вершина: '{curr_path}' --
[{ch}]--> '{node_to_str[-1]}'")
            else:
                print(f"        Переход по существующей вершине:
'{curr_path}' --[{ch}]--> '{node_to_str[trie[node][c_idx]]}'")
                node = trie[node][c_idx]
            output[node].append(idx + 1)
        print(f"        Шаблон '{pat}' завершён в вершине
'{node_to_str[node]}")

    q = deque()

    print("ПОСТРОЕНИЕ СУФФИКСНЫХ И КОНЕЧНЫХ ССЫЛОК.")
    for c_idx in range(sigma):
        if trie[0][c_idx] != -1:
            s = trie[0][c_idx]
            fail[s] = 0
            term[s] = 0
            q.append(s)
            print(f"        Суффиксная ссылка: '{node_to_str[s]}' -> '
(прямой ребёнок корня)")
```

```

        else:
            trie[0][c_idx] = 0

    while q:
        v = q.popleft()
        for c_idx in range(sigma):
            u = trie[v][c_idx]
            if u != -1:
                fail[u] = trie[fail[v]][c_idx]
                print(f"\n          Обработка          вершины
'{node_to_str[u]}':")
                print(f"          Суффиксная ссылка: '{node_to_str[u]}'
-> '{node_to_str[fail[u]]}'")

                if output[fail[u]]:
                    term[u] = fail[u]
                    print(f"          Конечная          ссылка:
'{node_to_str[u]}' -> '{node_to_str[term[u]]}'")
                else:
                    term[u] = term[fail[u]]
                    if term[u] == 0:
                        print(f"          Конечная          ссылка:
'{node_to_str[u]}' -> ''")
                    else:
                        print(f"          Конечная          ссылка:
'{node_to_str[u]}' -> '{node_to_str[term[u]]}' (унаследована)")

                q.append(u)
            else:
                trie[v][c_idx] = trie[fail[v]][c_idx]

    results = []
    node = 0

    print("ПОИСК В ТЕКСТЕ.")
    for i, ch in enumerate(T):
        c_idx = char_to_idx[ch]
        prev_state = node_to_str[node]
        node = trie[node][c_idx]
        print(f"\nПозиция {i + 1}, символ '{ch}': переход
'{prev_state}' -> '{node_to_str[node]}')

        u = node if output[node] else term[node]

        while u != 0:
            for pat_idx in output[u]:
                pos = i + 2 - len(patterns[pat_idx - 1])
                print(f"          Найден шаблон '{patterns[pat_idx -
1]}' (ID: {pat_idx}) в позиции {pos}")
                results.append((pos, pat_idx))
            u = term[u]

    results.sort()

    print("РЕЗУЛЬТАТ.")
    if results:
        for pos, pat_idx in results:

```

```

        print(f"{pos} {pat_idx}")
    else:
        print("(Вхождения отсутствуют)")

if __name__ == "__main__":
    solve()

```

Название файла: lb5.2.py

```

from collections import deque

def solve():
    T = input("Введите текст: ").strip().upper()
    P = input("Введите шаблон: ").strip().upper()
    wildcard_input = input().strip().upper()

    wildcard_char = wildcard_input[0]
    forbidden_char = wildcard_input[1]

    parts = []
    positions = []
    wildcard_positions = []
    current_part = ""

    print("РАЗБОР ШАБЛОНА НА ЧАСТИ")
    i = 0
    while i < len(P):
        if P[i] == wildcard_char:
            if current_part:
                parts.append(current_part)
                positions.append(i - len(current_part))
                print(f"    Завершена часть '{current_part}' на
позиции {i - len(current_part)}")
                current_part = ""
                wildcard_positions.append(i)
                print(f"    Символ джокера '{wildcard_char}' на позиции
{i}")
            i += 1
        else:
            current_part += P[i]
            i += 1

    if current_part:
        parts.append(current_part)
        positions.append(len(P) - len(current_part))
        print(f"    Завершена часть '{current_part}' на позиции
{len(P) - len(current_part)}")

    print(f"\nИтого частей: {len(parts)}")
    print(f"Позиции диких символов: {wildcard_positions}")

    if not parts and not wildcard_positions:
        print("Шаблон пуст")
        return

    alphabet = sorted(list(set(T + ''.join(parts))))
    char_to_idx = {ch: i for i, ch in enumerate(alphabet)}

```

```

sigma = len(alphabet)

trie = [[-1] * sigma]
fail = [0]
output = [[]]
term = [0]
node_to_str = [""]

print("ПОСТРОЕНИЕ БОРА.")
for idx, part in enumerate(parts):
    print(f"\nДобавляем часть '{part}' (ID: {idx}):")
    node = 0
    for ch in part:
        c_idx = char_to_idx[ch]
        curr_path = node_to_str[node]
        if trie[node][c_idx] == -1:
            trie[node][c_idx] = len(trie)
            trie.append([-1] * sigma)
            fail.append(0)
            output.append([])
            term.append(0)
            node_to_str.append(curr_path + ch)
            print(f"        Создана вершина: '{curr_path}' --
[{ch}]--> '{node_to_str[-1]}'")
        else:
            print(f"        Переход по существующей вершине:
'{curr_path}' --[{ch}]--> '{node_to_str[trie[node][c_idx]]}'")
            node = trie[node][c_idx]
            output[node].append(idx)
            print(f"        Часть '{part}' завершена в вершине
'{node_to_str[node]}'")

    q = deque()

print("ПОСТРОЕНИЕ СУФФИКСНЫХ И КОНЕЧНЫХ ССЫЛОК.")
for c_idx in range(sigma):
    if trie[0][c_idx] != -1:
        s = trie[0][c_idx]
        fail[s] = 0
        term[s] = 0
        q.append(s)
        print(f"        Суффиксная ссылка: '{node_to_str[s]}' -> ''
(прямой ребёнок корня)")
    else:
        trie[0][c_idx] = 0

while q:
    v = q.popleft()
    for c_idx in range(sigma):
        u = trie[v][c_idx]
        if u != -1:
            fail[u] = trie[fail[v]][c_idx]
            print(f"\n        Обработка вершины
'{node_to_str[u]}'")
            print(f"        Суффиксная ссылка: '{node_to_str[u]}'
-> '{node_to_str[fail[u]]}'")

```

```

        if output[fail[u]]:
            term[u] = fail[u]
            print(f"                                Конечная      ссылка:
'{node_to_str[u]}' -> '{node_to_str[term[u]]}'")
        else:
            term[u] = term[fail[u]]
            if term[u] == 0:
                print(f"                                Конечная      ссылка:
'{node_to_str[u]}' -> ''")
            else:
                print(f"                                Конечная      ссылка:
'{node_to_str[u]}' -> '{node_to_str[term[u]]}' (унаследована)")

        q.append(u)
    else:
        trie[v][c_idx] = trie[fail[v]][c_idx]

print("ПОИСК ЧАСТЕЙ В ТЕКСТЕ.")
found_parts = [[] for _ in range(len(T))]

node = 0
for i, ch in enumerate(T):
    c_idx = char_to_idx[ch]
    node = trie[node][c_idx]
    prev_state = node_to_str[node]
    print(f"\nПозиция      {i},      СИМВОЛ      '{ch}':      переход
'{prev_state}' -> '{node_to_str[node]}')

    u = node if output[node] else term[node]

    while u != 0:
        for part_idx in output[u]:
            start_pos_in_pattern = positions[part_idx]
            end_pos_in_text = i
            start_pos_in_text = end_pos_in_text -
len(parts[part_idx]) + 1
            pattern_start = start_pos_in_text -
start_pos_in_pattern
            print(f"      Найдена часть '{parts[part_idx]}'
(ID: {part_idx}) в тексте на позиции {start_pos_in_text}")
            if pattern_start >= 0 and pattern_start + len(P)
<= len(T):
                found_parts[pattern_start].append((part_idx,
start_pos_in_text))
                print(f"                                Кандидат на начало шаблона:
позиция {pattern_start}")
            else:
                print(f"                                Кандидат {pattern_start}
отброшен")
        u = term[u]

print("ПРОВЕРКА КОНДИДАТОВ")
results = []
pattern_len = len(P)

for pos in range(len(T) - pattern_len + 1):
    print(f"\nКандидат на позицию {pos}:")

```

```

        print(f"            Найдено частей в этой позиции:
{len(found_parts[pos])} из {len(parts)}")

        if len(found_parts[pos]) == len(parts):
            valid = True
            part_found = [False] * len(parts)

            for part_idx, part_start_in_text in found_parts[pos]:
                expected_start = pos + positions[part_idx]
                print(f"        Часть {part_idx} '{parts[part_idx]}':
найдена на {part_start_in_text}, ожидается {expected_start}")
                if part_start_in_text == expected_start:
                    part_found[part_idx] = True
                    print("        Позиция совпадает.")
                else:
                    valid = False
                    print("        Позиция не совпадает")
                    break

            if not (valid and all(part_found)):
                print(f"        Кандидат {pos} отброшен: не все части
на своих местах")
                continue

            wildcard_valid = True
            for w_pos in wildcard_positions:
                text_pos = pos + w_pos
                print(f"        Проверка символа джокера: позиция
{text_pos} в тексте = '{T[text_pos]}')
                if T[text_pos] == forbidden_char:
                    wildcard_valid = False
                    print(f"        Запретный символ -
'{forbidden_char}')
                    break

            if wildcard_valid:
                print(f"        Кандидат {pos} ПРИНЯТ")
                results.append(pos + 1)

    print("РЕЗУЛЬТАТ.")
    if results:
        for pos in results:
            print(pos)
    else:
        print("(Совпадений не найдено)")

if __name__ == "__main__":
    solve()

```